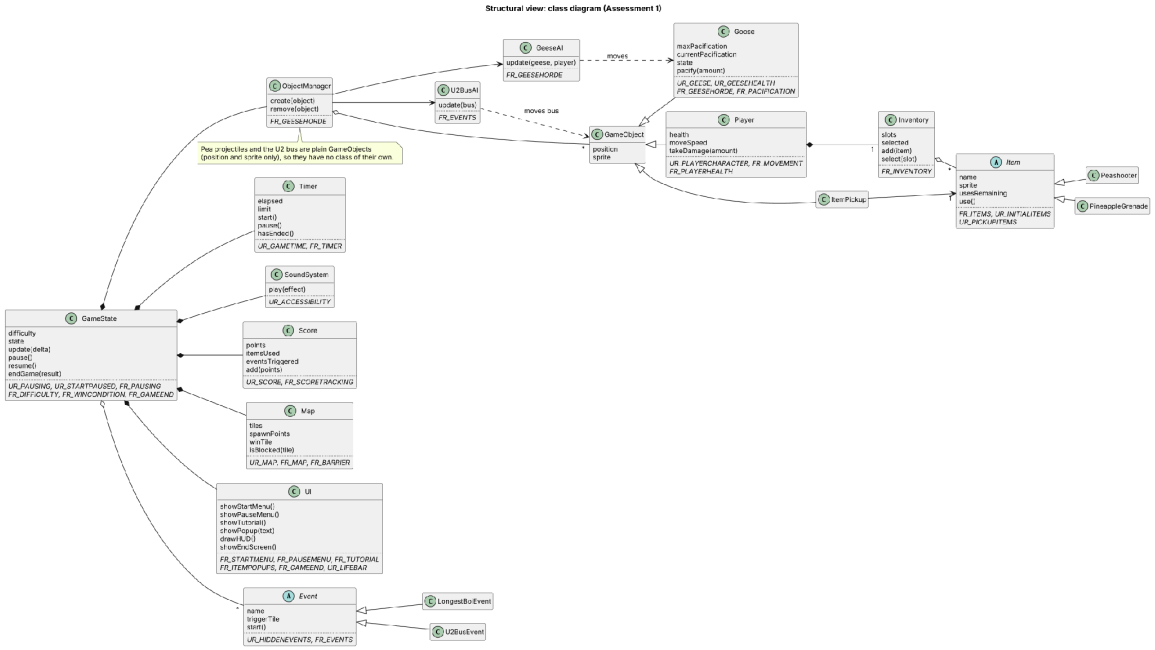
Architecture

The architecture is shown in UML using PlantUML (https://plantuml.com/). The class diagram shows the structure and three sequence diagrams show the behaviour. PlantUML is plain text, so the diagrams can be kept in version control and edited without a drawing tool (our preliminary diagrams were already written in it). The game is written in Java 17 using libGDX, and the diagrams only cover game-level classes. The italic lines in the class boxes and the brackets in the sequence diagrams are requirement IDs. These let each part be traced back to the requirements.

Architecture development stages:

1. A preliminary design is drawn in PlantUML: a hierarchy of seven classes (Player, GameState, SoundSystem, ObjectManager, GeeseAI, Timer, U2BusAI) and sequence diagrams for the game flow, tutorial flow, goose AI, four items (Peashooter, Popcorn, Translator, Pineapple Grenade) and two events (U2 Bus, Longest Boi).
2. The design is scoped to Assessment 1: 2 inventory items, 2 hidden events, a visible 5 minute timer, item use and event trackers, and a simple map.
3. Requirement IDs are added to the classes and messages, so changes can be traced between the requirements and the architecture.

Class Diagram



Note: Hard to view diagram, im looking for better solutions or i might just write the UML syntax for the diagram instead.

```
@startuml
title Structural view: class diagram (Assessment 1)
skinparam shadowing false
```

```
skinparam classAttributeIconSize 0
skinparam defaultFontName SansSerif
skinparam defaultFontSize 12
skinparam nodesep 25
skinparam ranksep 35
hide empty members
left to right direction

class GameState {
 difficulty
 state
 update(delta)
 pause()
 resume()
 endGame(result)
 ..
 //UR_PAUSING, UR_STARTPAUSED, FR_PAUSING//
 //FR_DIFFICULTY, FR_WINCONDITION, FR_GAMEEND//
}
class Timer {
 elapsed
 limit
 start()
 pause()
 hasEnded()
 ..
 //UR_GAMETIME, FR_TIMER//
}
class SoundSystem {
 play(effect)
 ..
 //UR_ACCESSIBILITY//
}
class Score {
 points
 itemsUsed
 eventsTriggered
 add(points)
 ..
 //UR_SCORE, FR_SCORETRACKING//
}
class Map {
 tiles
 spawnPoints
 winTile
 isBlocked(tile)
 ..
 //UR_MAP, FR_MAP, FR_BARRIER//
}
class UI {
 showStartMenu()
 showPauseMenu()
 showTutorial()
 showPopup(text)
 drawHUD()
 showEndScreen()
 ..
 //FR_STARTMENU, FR_PAUSEMENU, FR_TUTORIAL//
 //FR_ITEMPOPUPS, FR_GAMEEND, UR_LIFEBAR//
}
class ObjectManager {
 create(object)
 remove(object)
 ..
 //FR_GEESEHORDE//
}
class GameObject {
 position
 sprite
}
class Player {
 health
 moveSpeed
 takeDamage(amount)
 ..
```

```
  //UR_PLAYERCHARACTER, FR_MOVEMENT//
  //FR_PLAYERHEALTH//
}
class Goose {
  maxPacification
  currentPacification
  state
  pacify(amount)
  ..
  //UR_GEESE, UR_GEESEHEALTH//
  //FR_GEESEHORDE, FR_PACIFICATION//
}
class ItemPickup
class GeeseAI {
  update(geese, player)
  ..
  //FR_GEESEHORDE//
}
class U2BusAI {
  update(bus)
  ..
  //FR_EVENTS//
}
class Inventory {
  slots
  selected
  add(item)
  select(slot)
  ..
  //FR_INVENTORY//
}
abstract class Item {
  name
  sprite
  usesRemaining
  use()
  ..
  //FR_ITEMS, UR_INITIALITEMS//
  //UR_PICKUPITEMS//
}
class Peashooter
class PineappleGrenade
abstract class Event {
  name
  triggerTile
  start()
  ..
  //UR_HIDDENEVENTS, FR_EVENTS//
}
class U2BusEvent
class LongestBoiEvent

GameState *-- Timer
GameState *-- SoundSystem
GameState *-- Score
GameState *-- Map
GameState *-- UI
GameState *-- ObjectManager
GameState o-- "*" Event

ObjectManager o-- "*" GameObject
ObjectManager --> GeeseAI
ObjectManager --> U2BusAI

GameObject <|-- Player
GameObject <|-- Goose
GameObject <|-- ItemPickup

Player *-- "1" Inventory
Inventory o-- "*" Item
ItemPickup --> "1" Item

Item <|-- Peashooter
Item <|-- PineappleGrenade
Event <|-- U2BusEvent
```

```
Event <|-- LongestBoiEvent

GeeseAI ..> Goose : moves
U2BusAI ..> GameObject : moves bus

note bottom of ObjectManager
  Pea projectiles and the U2 bus are plain GameObjects
  (position and sprite only), so they have no class of their own.
end note
@enduml
```

Representation of the game's systems. GameState controls the game and owns the Timer, SoundSystem, Score, Map, UI and ObjectManager. ObjectManager holds every game object and uses GeeseAI and U2BusAI to move them. The Player owns an Inventory of Items.

Table of Classes

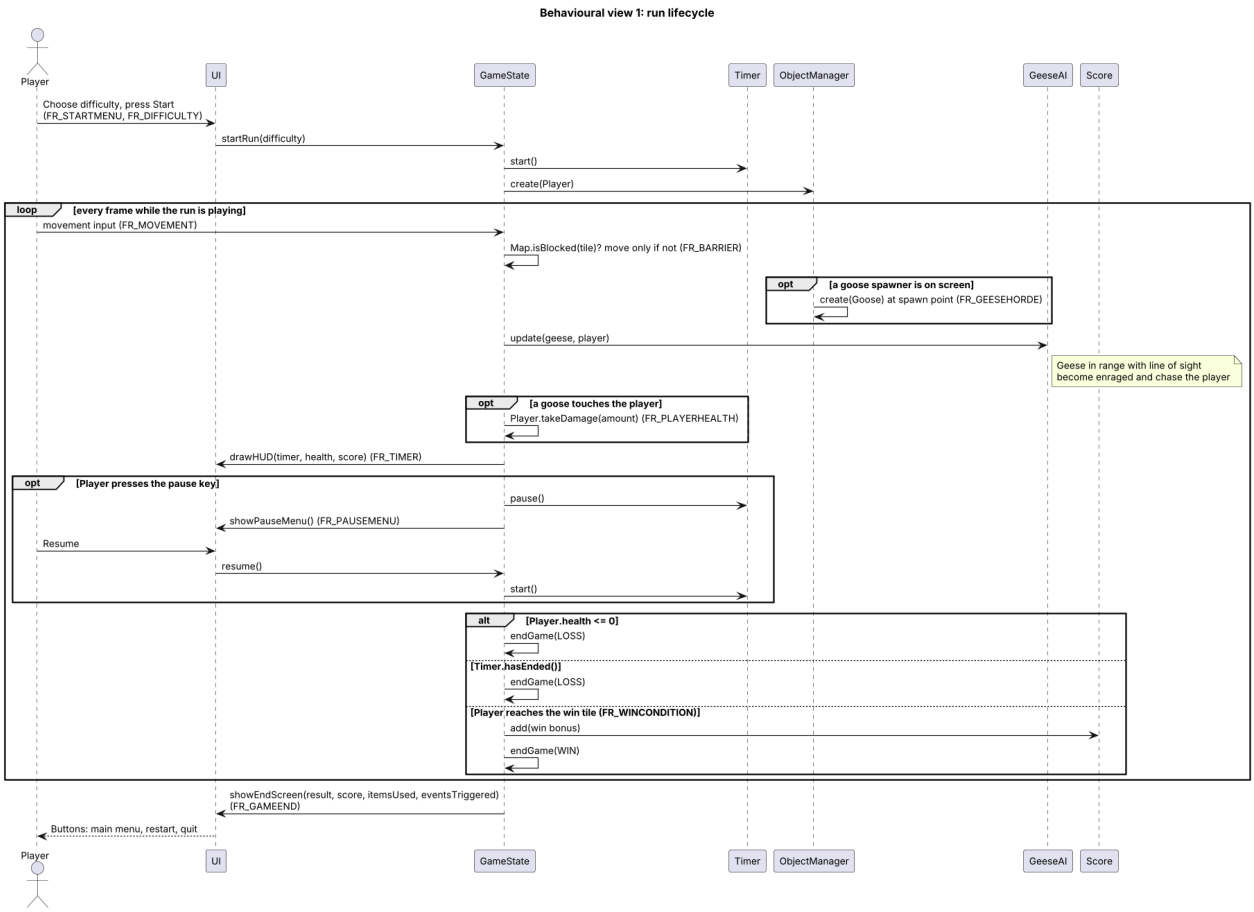
Name	Description	Implementation	Requirement Link
GameState	Single controller for the game. Pausing, difficulty, the win tile and the end of a run all affect every other part, so they are kept in one place.	Stores difficulty and state (start menu, playing, paused, ended). update(delta) each frame, pause(), resume(), endGame(result).	UR_PAUSING, UR_STARTPAUSED, UR_DIFFICULTIES, UR_WINCONDITION, FR_PAUSING, FR_DIFFICULTY, FR_WINCONDITION, FR_GAMEEND
Timer	Independent timer, so pausing the game can stop it. Counts the 5 minute run.	Stores elapsed and limit. start(), pause(), hasEnded(). The HUD draws the time left as a shrinking bar.	UR_GAMETIME, FR_TIMER
SoundSystem	Plays sound effects, so information is also given through audio.	play(effect)	UR_ACCESSIBILITY, FR_TIMER, QR_INCLUSIVITY
Score	One score for the run, with counters for items used and events triggered.	Stores points, itemsUsed and eventsTriggered. add(points).	UR_SCORE, FR_SCORETRACKING
Map	Grid of tiles holding barriers, goose spawn points and the win tile.	Stores tiles, spawnPoints and winTile. isBlocked(tile).	UR_MAP, FR_MAP, FR_BARRIER
UI	Start menu, pause menu, tutorial, HUD (timer bar,	showStartMenu(), showPauseMenu(),	UR_TUTORIAL, UR_LIFEBAR,

Name	Description	Implementation	Requirement Link
	health, hotbar), popups and end screen. One class for every screen to keep it simple.	showTutorial(), showPopup(text), drawHUD(), showEndScreen().	UR_ACCESSIBILITY, FR_STARTMENU, FR_PAUSEMENU, FR_TUTORIAL, FR_ITEMPOPUPS, FR_GAMEEND, FR_TIMER, FR_INVENTORY, FR_PLAYERHEALTH, QR_POPUPS, QR_HOVERBUTTON, QR_UNDERSTANDING, QR_INCLUSIVITY, QR_UNINTRUSIVEAMMO
ObjectManager	Creates and removes game objects. Pea projectiles and the U2 bus only need a position and a sprite, so they are plain GameObjects.	create(object), remove(object). Uses GeeseAI and U2BusAI.	UR_GEESE, FR_GEESEHORDE, QR_GEESEAGGRESSION
GameObject, Player	The player character. Owns the Inventory.	Stores position and sprite, plus health and moveSpeed. takeDamage(amount).	UR_PLAYERCHARACTER, FR_MOVEMENT, FR_PLAYERHEALTH
Goose	Hostile goose with pacification instead of health.	Stores maxPacification, currentPacification and state. pacify(amount). At the maximum the goose stops chasing and is removed.	UR_GEESE, UR_GEESEHEALTH, UR_NOVIOLENCE, FR_GEESEHORDE, FR_PACIFICATION, QR_GEESEDIRECTION
GeeseAI	Goose movement is kept apart from goose data, so values such as goose speed can be changed in one place (game difficulty risk).	update(geese, player). Geese in range with line of sight become enraged and chase the player.	FR_GEESEHORDE, QR_GEESEHINDERANCE
U2BusAI	Moves the U2 bus across the screen towards geese.	update(bus)	FR_EVENTS

Name	Description	Implementation	Requirement Link
Inventory	Hotbar of items with one selected at a time.	Stores slots and selected. add(item), select(slot).	UR_INITIALITEMS, UR_PICKUPITEMS, FR_INVENTORY
Item, Peashooter, PineappleGrenade	Abstract item with a sprite, a number of uses and an effect. Peashooter is an initial item with unlimited uses. Pineapple Grenade is a single use pickup that pacifies geese in an area.	Stores name, sprite and usesRemaining. use().	UR_INITIALITEMS, UR_PICKUPITEMS, FR_ITEMS, FR_PACIFICATION
ItemPickup	Item lying on the map. Picking it up adds it to the Inventory and shows a popup.	Holds one Item.	UR_PICKUPITEMS, FR_ITEMS, FR_ITEMPOPUPS, QR_ITEMLOCATION
Event, U2BusEvent, LongestBoiEvent	Abstract event with a trigger tile and an effect. New events can be added without changing GameState.	Stores name and triggerTile. start().	UR_HIDDENEVENTS, FR_EVENTS

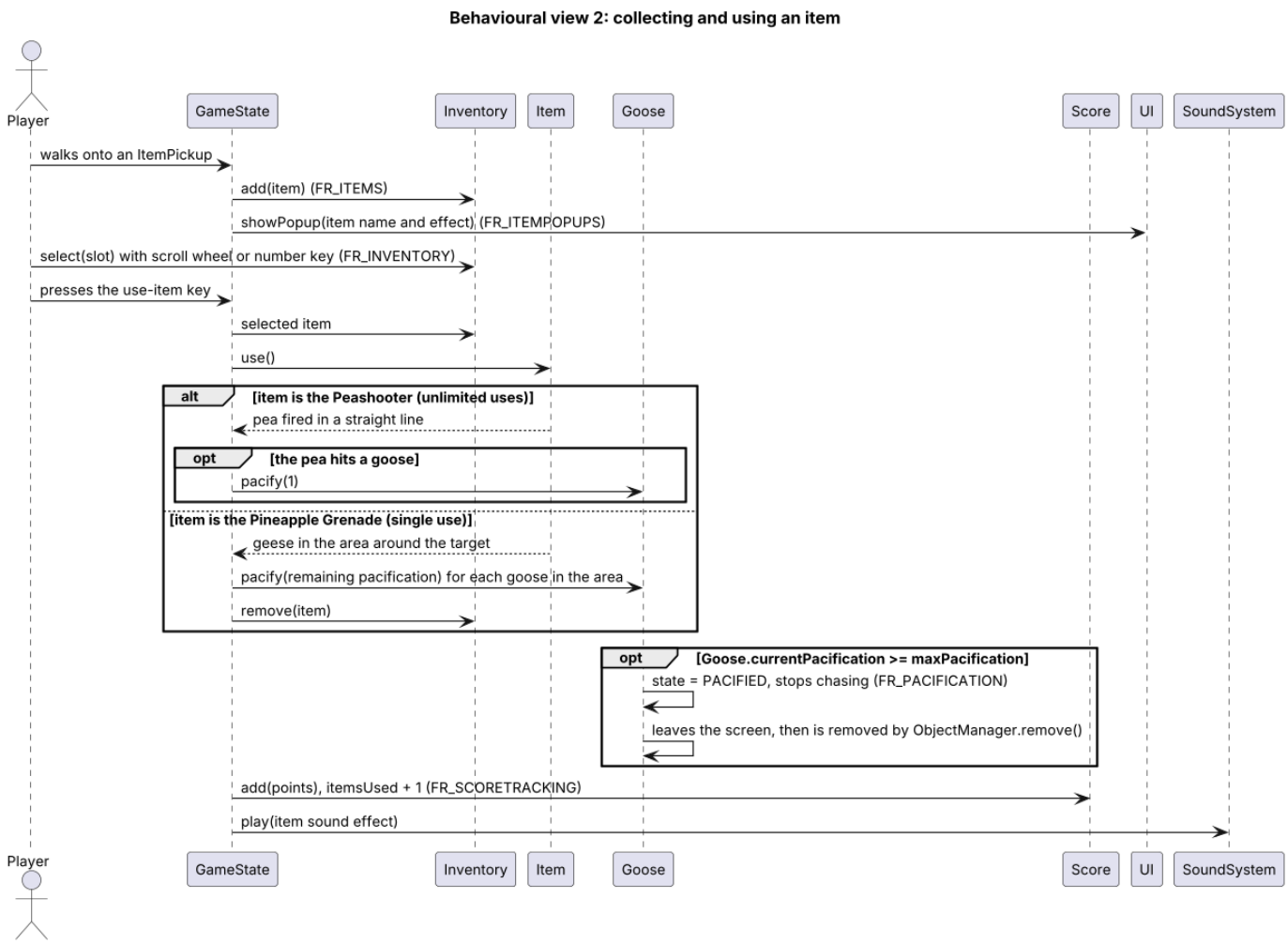
UR_NOCOPYRIGHTINFRINGEMENT applies to assets and licences rather than structure. UR_LOWDIFF and QR_EASY are met by the values chosen (goose speed, goose damage, pacification amounts) rather than by a class.

Run Lifecycle



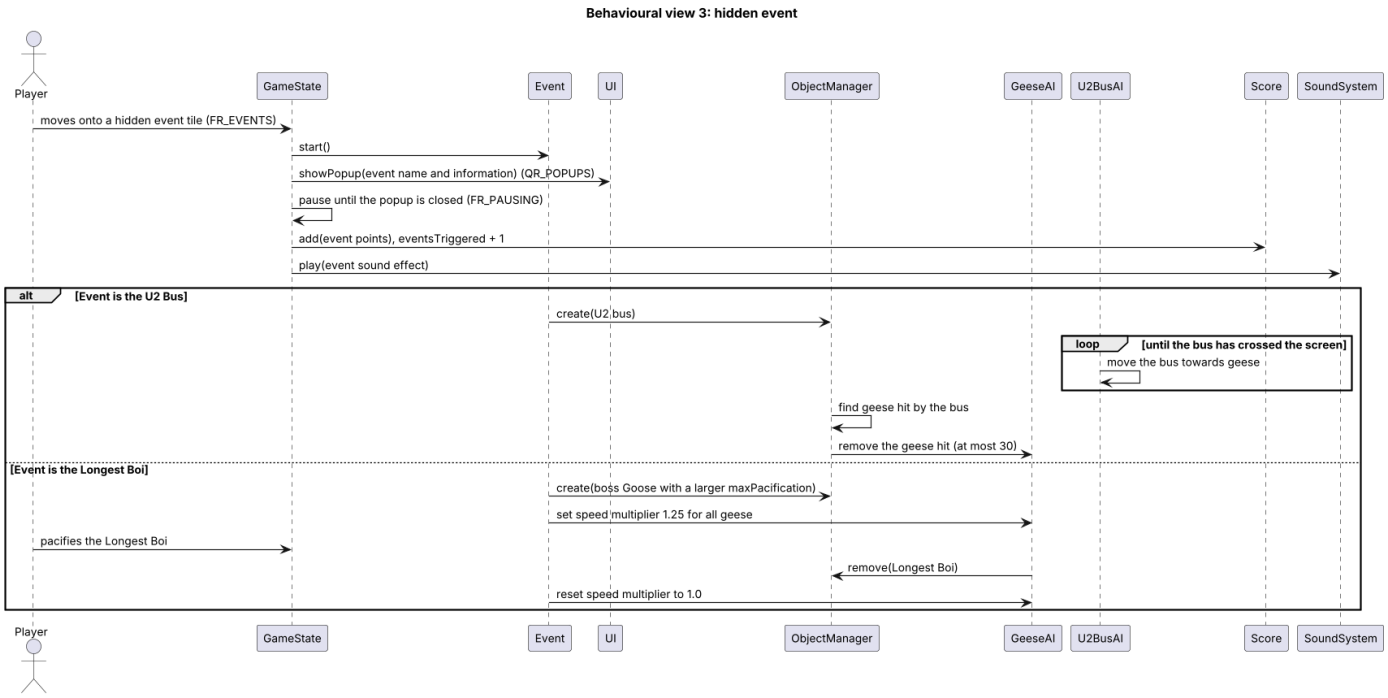
Representation of a run from the start menu to the end screen. Each frame GameState moves the player, lets ObjectManager spawn geese and GeeseAI move them, applies goose contact damage and redraws the HUD. The run ends when the player's health reaches 0 or the Timer ends (a loss), or when the player reaches the win tile (a win with a score bonus). The end screen shows the score and the item and event counts.

Collecting and Using an Item



Simple representation of how items are picked up and used. The Peashooter is an initial item with unlimited uses, and the Pineapple Grenade is a single use pickup. Both pacify geese by feeding them. A goose whose current pacification reaches its maximum stops chasing the player and leaves the screen. Each use adds points, adds to the items used count and plays a sound effect.

Hidden Event



Simple representation of how a hidden event works. Stepping on a hidden tile pauses the game, shows a popup with the event's name and information, and adds points and one to the events triggered count. The U2 Bus then removes up to 30 geese. The Longest Boi adds a larger boss goose and gives all geese 1.25x movement speed until the boss is pacified.